17.8.3 Γενικευμένη συνάρτηση υπολογισμού γινομένου

Να υλοποιηθεί συνάρτηση product που λαμβάνει ως παράμετρο λίστα ενός οποιουδήποτε αριθμητικού τύπου και επιστρέφει το γινόμενο των στοιχείων της λίστας ως double. Προσθέστε main στην οποία ελέγξτε την product με ArrayList<Integer> και LinkedList<Double>.

Βιβλιογραφία

[1] “Generics, Inheritance, and Subtypes (The Java™ Tutorials > Learning the Java Language > Generics (Updated)).” https://docs.oracle.com/javase/tutorial/java/generics/inheritance.html (accessed Nov. 19, 2021).

Κεφάλαιο 18

Σύνοψη
Στην ενότητα αυτή παρουσιάζονται οι εκφράσεις λάμδα μαζί με τις λειτουργικές διεπαφές, οι αναφορές μεθόδων καθώς και η απομνημόνευση, μία τεχνική που περιορίζει των αριθμό των κλήσεων στις αναδρομικές συναρτήσεις και κάνει τον κώδικα πιο αποτελεσματικό.

Προαπαιτούμενη γνώση
Η ενσωμάτωση, η κληρονομικότητα και ο πολυμορφισμός στο αντικειμενοστρεφές μοντέλο με Java, οι αφηρημένες κλάσεις καθώς και οι βασικές προκαθορισμένες κλάσεις, δηλαδή οι String, StringBuilder, ArrayList, LinkedList, Stack, PriorityQueue, HashSet, HashMap, HashTree, LocalDate, Localtime, Period και Duration. Οι διεπαφές στην Java και οι γενικεύσεις.

Λέξεις κλειδιά
Λειτουργικές διεπαφές (functional interfaces), Εκφράσεις λάμδα (Lamda expressions), Αναφορές μεθόδων (method references), Απομνημόνευση (memoization).

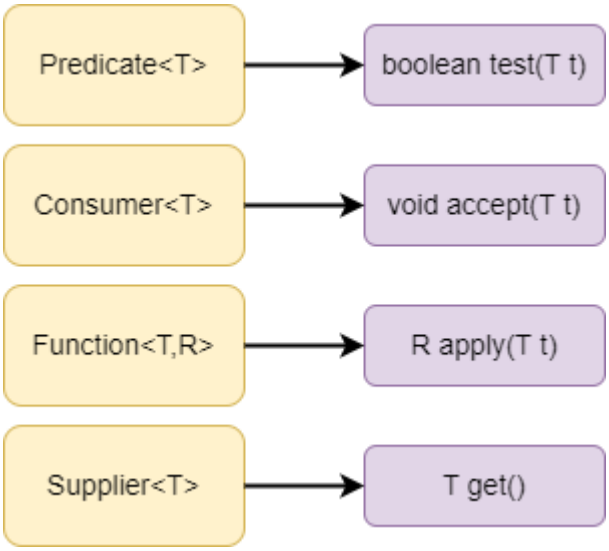
18 Ειδικά θέματα

18.1 Εκφράσεις Λάμδα

Συχνά χρειάζεται να περάσουμε ως παράμετρο σε μία συνάρτηση ένα αντικείμενο. Σε κάποιες περιπτώσεις είναι αντικείμενο μιας κλάσης που υλοποιεί μία διεπαφή. Σύμφωνα με όσα έχουμε ήδη μάθει, έχουμε δύο δυνατότητες. Να ορίσουμε την κλάση που υλοποιεί τη διεπαφή και στη συνέχεια το αντικείμενο ή να δημιουργήσουμε ένα αντικείμενο ανώνυμης κλάσης. Όμως αρκετά συχνά η διεπαφή έχει μία μόνο αφηρημένη μέθοδο. Επομένως, σε αυτές τις περιπτώσεις, το μόνο που είναι πράγματι αναγκαίο να κάνουμε είναι να ορίσουμε τον κώδικα της αφηρημένης μεθόδου. Θα ήταν μεγάλη ευκολία αν μπορούσαμε να περάσουμε ως παράμετρο κατευθείαν τον κώδικα που υλοποιεί την αφηρημένη μέθοδο και αποφεύγαμε τη δήλωση επώνυμης ή και ανώνυμης κλάσης. Αυτήν ακριβώς τη λειτουργικότητα μας παρέχουν οι εκφράσεις Λάμδα (Lamda expressions) που προστέθηκαν στην έκδοση 8 της Java.

18.1.1 Λειτουργικές διεπαφές

Λειτουργικές διεπαφές (functional interfaces) είναι αυτές που διαθέτουν μία μόνο αφηρημένη μέθοδο [1]. Βεβαίως, μια λειτουργική διεπαφή μπορεί να περιλαμβάνει πολλές στατικές ή προκαθορισμένες μεθόδους. Η Java διαθέτει ένα μεγάλο πλήθος προκαθορισμένων λειτουργικών διεπαφών. Ειδικά το πακέτο java.util.function αποτελείται αποκλειστικά από λειτουργικές διεπαφές, ενώ υπάρχουν πολλές τέτοιες διεπαφές διασκορπισμένες και στα υπόλοιπα πακέτα. Φυσικά, λειτουργικές διεπαφές μπορούν να οριστούν και από τον χρήστη.
Στο σχήμα 18.1 παρουσιάζονται τέσσερις βασικές προκαθορισμένες λειτουργικές διεπαφές που ανήκουν στο πακέτο java.util.function:



Σχήμα 18.1 *Τέσσερις βασικές λειτουργικές διεπαφές*

Η διεπαφή Predicate<T> δηλώνει την αφηρημένη μέθοδο boolean test(T t). Η Consumer<T> την αφηρημένη μέθοδο void accept(T t). Η Function<T,R> την R apply(T t) και η Supplier<T> την T get().
Στη συνέχεια θα δούμε πώς μπορεί να αξιοποιηθούν αυτές οι διεπαφές σε συνδυασμό με τις εκφράσεις lamda, ξεκινώντας από την Predicate<T>.
Ας υποθέσουμε πως χρειαζόμαστε να τυπώνουμε λίστες από αντικείμενα Person στις οποίες θα περιλαμβάνονται μόνο όσα πρόσωπα πληρούν κάποια ηλικιακά κριτήρια. Πιο συγκεκριμένα, έστω ότι θέλουμε να τυπώσουμε μία λίστα με όσα πρόσωπα είναι ηλικίας άνω των 50 ετών και μία λίστα με όσα πρόσωπα είναι από 40 έως 55 ετών. Μπορούμε να φτιάξουμε μία συνάρτηση που θα παίρνει ως παράμετρο μία λίστα με το σύνολο των προσώπων και ένα αντικείμενο Predicate<Person>. Εφόσον ένα πρόσωπο θα περνά επιτυχώς την test της Predicate θα τυπώνεται. Στον κώδικα 18.1 παρουσιάζεται μία τέτοια συνάρτηση:

```
static void prt(List<Person> list, Predicate<Person> choice) {
    Iterator<Person> it = list.iterator();
    while (it.hasNext()) {
        Person p = it.next();
        if (choice.test(p)) {
            System.out.println(p + " " + p.getAge());
        }
    }
}
```

Κώδικας 18.1 *Εκτύπωση λίστας προσώπων με κριτήρια*

Σύμφωνα με όσα γνωρίζουμε για να καλέσουμε την ptr ώστε να εκπληρώσουμε τον σκοπό μας χρειαζόμαστε δύο κλάσεις όπως δείχνει ο κώδικας 18.2.

```
class SelectorGreater50 implements Predicate<Person> {

    @Override
    public boolean test(Person t) {
        return t.getAge() > 50;
    }
}

class SelectorBetween40and55 implements Predicate<Person> {

    @Override
    public boolean test(Person t) {
       return t.getAge() >=40 && t.getAge()<=55;
    }
```

```
    }
```

Κώδικας 18.2 *Δύο κλάσεις που υλοποιούν τη διεπαφή Predicate*

Έχοντας στη διάθεσή μας τους κώδικες 18.1 και 18.2, μπορούμε να εκπληρώσουμε τον στόχο μας όπως δείχνει ο κώδικας 18.3.

```
public static void main(String[] args) {
    List<Person> list = loadData();
    prt(list, new SelectorGreater50());
    System.out.println("--------------------");
    prt(list, new SelectorBetween40and55());
}
```

Κώδικας 18.3 *Εκτύπωση προσώπων άνω των 50 ετών και εκτύπωση προσώπων από 40 έως και 55 ετών.*

Προσέξτε τώρα τα εξής: Καταρχάς, η διεπαφή Predicate είναι λειτουργική, δηλαδή διαθέτει μία μόνο αφηρημένη μέθοδο, την test. Επομένως, για να ορίσουμε μία κλάση που υλοποιεί τη διεπαφή εκείνο που είναι αναγκαίο είναι ο ορισμός της test. Στη μία περίπτωση ο ορισμός της test δίνεται από τον κώδικα

```
return t.getAge() > 50;
```

και στην άλλη περίπτωση από τον κώδικα

```
return t.getAge() >=40 && t.getAge()<=55;
```

Γιατί όμως να είμαστε αναγκασμένοι να ορίσουμε τις κλάσεις SelectorGreater50 και SelectorBetween40and55, αφού η μόνη πληροφορία που προσθέτουμε σε κάθε κλάση περιορίζεται στον αντίστοιχο κώδικα της test; Εδώ είναι που μας βοηθούν οι εκφράσεις λάμδα κάνοντας τον κώδικά μας συνοπτικό και κομψό, ενώ αυξάνουν την παραγωγικότητά μας. Οι εκφράσεις λάμδα λοιπόν μας επιτρέπουν να περάσουμε στη θέση ενός αντικειμένου λειτουργικής διεπαφής μόνο τον κώδικα που υλοποιεί τη μοναδική αφηρημένη μέθοδο της διεπαφής.
Έτσι, ο κώδικας 18.3 μπορεί να μεταγραφεί με τη χρήση των εκφράσεων λάμδα όπως δείχνει ο κώδικας 18.4.

```
public static void main(String[] args) {
    List<Person> list = loadData();
    prt(list, t->t.getAge()>50);
    System.out.println("----------");
    prt(list, t->t.getAge()>=40 && t.getAge()<=55);
}
```

Κώδικας 18.4 *Χρήση των εκφράσεων λάμδα*

Στην πρώτη κλήση της prt στον κώδικα 18.4, δίνουμε τον κώδικα που υλοποιεί την test για ηλικίες άνω των 50 ετών. Η test γνωρίζουμε πως απαιτεί ως παράμετρο ένα Person, γεγονός που είναι γνωστό και στον μεταγλωττιστή, επομένως δεν είναι αναγκαίο να ορίσουμε τον τύπο της παραμέτρου. Επειδή όμως είναι αναγκαίο να αναφερθούμε σε αυτήν, θα πρέπει να της δώσουμε ένα όνομα. Το όνομα αυτό προηγείται του συμβόλου ->, ενώ ο κώδικας που ακολουθεί το σύμβολο -> είναι ο κώδικας που υλοποιεί την αφηρημένη μέθοδο.
Με τη βοήθεια των εκφράσεων λαμδα είναι εύκολο να τυπώσουμε λίστες προσώπων με οποιαδήποτε κριτήρια ηλικιακά ή μη. Για παράδειγμα, μπορούμε να χρησιμοποιήσουμε την prt για να τυπώσουμε όλα τα πρόσωπα της αρχικής λίστας όπως δείχνει ο κώδικας που ακολουθεί:

```
prt(list, p->true);
```

ή να τυπώσουμε όλα τα πρόσωπα που έχουν επώνυμο Smith

```
prt(list, p->p.getsName().equals("Smith"));
```

Στη συνέχεια δίνουμε παραδείγματα αξιοποίησης των υπόλοιπων τριών λειτουργικών διεπαφών που αναφέρονται σε αυτήν την ενότητα.
Όπως θα έγινε αντιληπτό, η διεπαφή Predicate έχει ως στόχο να λαμβάνει μία παράμετρο, να την αξιολογεί και να επιστρέφει true ή false ανάλογα με κάποια κριτήρια. Αντίθετα, η διεπαφή Consumer έχει ως στόχο να επεξεργάζεται την παράμετρο της αφηρημένης μεθόδου της. Ο κώδικας 18.5 δίνει ένα σχετικό παράδειγμα:

```
import java.util.Iterator;
import java.util.List;
import java.util.function.Consumer;

public class Consumers {

    static void process(List<Person> list, Consumer<Person> consumer) {
        Iterator<Person> it = list.iterator();
        while (it.hasNext()) {
            Person p = it.next();
            consumer.accept(p);
        }
    }

    public static void main(String[] args) {
        List<Person> list = loadData();
        prt(list, p -> true);
        System.out.println("-----------------------");
        process(list, p -> p.setBirthday(p.getBirthday().minusYears(20)));
        prt(list, p -> true);

    }
}
```

Κώδικας 18.5 *Παράδειγμα χρήσης της Consumer*

Η συνάρτηση process του κωδικα 18.5 διασχίζει μια λίστα από αντικείμενα Person και σε κάθε αντικείμενο εφαρμόζει την επεξεργασία της Consumer. Η κλήση της στην main έχει ως αποτέλεσμα τη μεταβολή της ημερομηνίας γέννησης για όλα τα πρόσωπα της λίστας. Πιο συγκεκριμένα, μετά την κλήση της process στην main, όλα τα πρόσωπα της λίστας εμφανίζονται γηραιότερα κατά 20 έτη.
Ακολουθεί παράδειγμα χρήσης της διεπαφής Function:

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.function.Function;
import static tmp.FunctionalInterfaces.Predicates.prt;

public class Functions {

    static List<Person> getOlders(List<Person> list,
Function<Person,Boolean> f) {
        List<Person> rVal=new ArrayList<>();
        Iterator<Person> it=list.iterator();
        while (it.hasNext()) {
            Person p=it.next();

            if (f.apply(p)) {
               rVal.add(p);
            }
        }
        return rVal;
    }
```

```
    public static void main(String[] args) {
        List<Person> list = loadData();
        Function<Person, Boolean> ageLimit=p->p.getAge()>50;
        prt(list,p->true);
        System.out.println("-----------------");
        List<Person> olders=getOlders(list, ageLimit);
        prt(olders,p->true);
    }
}
```

Κώδικας 18.6 *Παράδειγμα χρήσης της Function*

Η Function είναι παραμετροποιημένη ως προς δύο τύπους. Η αφηρημένη μέθοδός της, apply, λαμβάνει τον ένα τύπο ως παράμετρο και επιστρέφει τον άλλο. Στον κώδικα 18.6, χρησιμοποιείται Function<Person, Boolean>. Στην main με τον ορισμό της ageLimit, επιδεικνύεται η δυνατότητα ορισμού εκφράσεων λάμδα ως απλές μεταβλητές. Η ageLimit περνά στην getOlders. Η getOlders προσθέτει κάθε Person για το οποίο η apply επιστρέφει true στην λίστα rVal. Τελικά η getOlders επιστρέφει μια λίστα με όλα τα πρόσωπα άνω των 50 ετών. Στη συνέχεια, στον κώδικα 18.7, με τη βοήθεια της διεπαφής Supplier δημιουργούμε μία μεταβλητή ικανή να παράγει τυχαίους ακέραιους:

```
import java.util.Random;
import java.util.function.Supplier;

public class Suppliers {

    public static void main(String[] args) {
        Supplier<Integer> gen = () -> new Random().nextInt();
        for (int i = 0; i < 10; i++) {
            System.out.println(gen.get());
        }
    }
}
```

Κώδικας 18.7 *Παράδειγμα χρήσης της Supplier*

Προσέξτε πως καθώς η αφηρημένη μέθοδος get δεν δέχεται παραμέτρους, στον ορισμό της έκφρασης λάμδα έχουμε δώσει κενή λίστα παραμέτρων.

18.1.2 Το συντακτικό των εκφράσεων λάμδα

Μία έκφραση λάμδα συντάσσεται ως ακολούθως:

```
λίστα παραμέτρων βέλος κώδικας
```

Η λίστα παραμέτρων μπορεί να είναι κενή ή να περιλαμβάνει μία ή περισσότερες παραμέτρους. Στην περίπτωση που περιλαμβάνει μία μόνο παράμετρο, οι παρενθέσεις μπορούν να παραλειφθούν.

Ο κώδικας μπορεί να είναι μία απλή έκφραση ή ένα μπλοκ κώδικα. Στην περίπτωση που είναι μπλοκ κώδικα και εφόσον η αφηρημένη μέθοδος δεν είναι void, η επιστροφή θα πρέπει να γίνει με χρήση της δεσμευμένης λέξης return.

Οι εκφράσεις λάμδα είναι στην ουσία ένα είδος ανώνυμων συναρτήσεων [2]. Προσέξτε το επόμενο παράδειγμα όπου ταξινομούμε μία λίστα από πρόσωπα πρώτα ως προς το id τους και μετά ως προς επώνυμο και όνομα.

```
import java.util.List;

public class ComparatorLamda {

    public static void main(String[] args) {
        List<Person> list = PolymorphismDemo.loadData();
        list.sort((p1, p2) -> {
```

```
            int id1 = Integer.decode(p1.getId());
            int id2 = Integer.decode(p2.getId());

            return id1 - id2;
        });
        prt(list, p -> true);
        System.out.println("--------------------");
        list.sort((p1, p2) -> {
            String sName1 = p1.getsName();
            String sName2 = p2.getsName();
            String fName1 = p1.getfName();
            String fName2 = p2.getfName();
            int rVal = sName1.compareTo(sName2);
            if (rVal == 0) {
                return fName1.compareTo(fName2);
            }
            return rVal;
        });
        prt(list, p -> true);
    }
```

Κώδικας 18.8 *Ταξινόμηση με εκφράσεις λάμδα*

18.1.3 Αναφορές μεθόδων

Εφόσον έχουμε στη διάθεσή μας μία μέθοδο που είναι ταυτόσημη ως προς τη λειτουργικότητα με τον κώδικα μίας έκφρασης λάμδα, τότε η έκφραση λάμδα μπορεί να αντικατασταθεί από την αναφορά της μεθόδου (method reference) [3].

```
import java.util.List;

public class MethodReferences {

    static boolean isOlder(Person p) {
        return p.getAge() > 50;
    }

    static boolean between40And55(Person p) {
        return p.getAge() >= 40 && p.getAge() <= 55;
    }

    public static void main(String[] args) {
        List<Person> list = loadData();
        prt(list, MethodReferences::isOlder);
        System.out.println("----------");
        prt(list, MethodReferences::between40And55);
    }
}
```

Κώδικας 18.9 *Χρήση αναφορών μεθόδων*

Ο κώδικας 18.9 περιλαμβάνει δύο boolean μεθόδους, την isOlder και την between40And55. Με δεδομένες αυτές τις μεθόδους, ο κώδικας 18.4 έχει μετασχηματιστεί στην main του κώδικα 18.9. Με άλλα λόγια, η έκφραση λάμδα

```
t->t.getAge()>50
```

αντικαταστάθηκε από την αναφορά

```
MethodReferences::isOlder
```

και η έκφραση

```
t->t.getAge()>=40 && t.getAge()<=55
```

αντικαταστάθηκε από την αναφορά

```
MethodReferences::between40And55
```

Στο παράδειγμα αυτό οι εκφράσεις λάμδα απαιτούν ως παράμετρο ένα αντικείμενο τύπου Person, την ίδια παράμετρο απαιτούν και οι αναφορές μεθόδων που αντικατέστησαν τις εκφράσεις λάμδα. Επίσης τόσο οι εκφράσεις λάμδα και οι αναφορές μεθόδων επιστρέφουν boolean τιμή. Είναι αυτονόητο πως για να επιτρέψει ο μεταγλωττιστής αντικατάσταση μιας έκφρασης λάμδα, θα πρέπει τόσο οι παράμετροί της όσο και η τιμή επιστροφής της να είναι συμβατές με αυτές της μεθόδου αναφοράς.
Στο παράδειγμα αυτό χρησιμοποιήθηκαν στατικές μέθοδοι. Μπορεί βέβαια να χρησιμοποιηθούν και μέθοδοι στιγμιότυπου. Για παράδειγμα, η πρώτη γραμμή της main στον κώδικα 18.7, δηλαδή ο κώδικας

```
Supplier<Integer> gen = () -> new Random().nextInt();
```

είναι ισοδύναμος με τον κώδικα

```
Supplier<Integer> gen = new Random()::nextInt;
```

Ειδικά, οι αναφορές σε δημιουργούς γίνονται με τη χρήση της δεσμευμένης λέξης new. Για παράδειγμα, ο κώδικας

```
Supplier<String> newString = () -> new String();
```

μπορεί να αντικατασταθεί από τον κώδικα

```
Supplier<String> newString = String::new;
```

Όταν όμως ο δημιουργός διαθέτει παραμέτρους, τότε δεν μπορεί να χρησιμοποιηθεί η διεπαφή Supplier, καθώς η μέθοδος get είναι χωρίς παραμέτρους. Μπορεί όμως να χρησιμοποιηθεί η διεπαφή Function, όπως επιδεικνύει ο κώδικας που ακολουθεί:

```
Function<String, String> s=String::new;
System.out.print(s.apply("Hello"));
System.out.println(s.apply(" Mike"));
```

18.2 Απομνημόνευση

Η απομνημόνευση (memoization) είναι μία τεχνική που εφαρμόζεται σε συνδυασμό με την αναδρομή με σκοπό να περιορίσει το πλήθος των αναδρομικών κλήσεων όπου αυτό είναι δυνατό.
Ας πάρουμε για παράδειγμα τον υπολογισμό του ν-οστού όρου της ακολουθίας Fibonacci. Ο κώδικας 9.4 υπολογίζει αναδρομικά τον ν-οστό όρο. Επίσης, το σχήμα 9.3 μας δίνει μία εικόνα των αναδρομικών κλήσεων. Όπως φαίνεται στο σχήμα 9.3, κατά τον υπολογισμό του όρου με ν=4, η κλήση για n=2 επαναλαμβάνεται δύο φορές. Αν επιχειρήσουμε να υπολογίσουμε τον 20ό όρο της ακολουθίας και μετρήσουμε τις αναδρομικές κλήσεις (Ενότητα 9.7, Άσκηση 5) θα διαπιστώσουμε πως απαιτούνται 21891 αναδρομικές κλήσεις. Αν όμως βρούμε έναν τρόπο ώστε οι τιμές της ακολουθίας να υπολογίζονται μόνο μία φορά η κάθε μία, οι αναδρομικές κλήσεις μειώνονται δραματικά.

```
import java.util.HashMap;

public class Fibonacci {
```

```
    static long cnt1 = 0;
    static long cnt2 = 0;

    static long fibonacciR(int n) {
        cnt1++;
        if (n == 0 || n == 1) {
            return 1;
        } else {
            return fibonacciR(n - 1) + fibonacciR(n - 2);
        }
    }

    static long fibonacciDP(int n, HashMap<Integer, Long> mem) {
        cnt2++;
        if (n == 0 || n == 1) {
            return 1;
        }
        try {
            return mem.get(n);
        } catch (Exception e) {
            long rslt = fibonacciDP(n - 1, mem) + fibonacciDP(n - 2, mem);
            mem.put(n, rslt);
            return rslt;
        }
    }

    public static void main(String[] args) {
        System.out.println("fib(" + 20 + ")=" + fibonacciR(20));
        System.out.println("----------------------");
        System.out.println("fib(" + 20 + ")=" + fibonacciDP(20, new
HashMap<>()));
        System.out.println(cnt1 + " " + cnt2);
    }
}
```

Κώδικας 18.10 *Η αναδρομική συνάρτηση Fibonacci χωρίς απομνημόνευση και με απομνημόνευση*

Ο κώδικα 18.10 παρουσιάζει δύο εκδόσεις της Fibonacci. Και οι δύο εκδόσεις είναι αναδρομικές. Η συνάρτηση FibonacciR είναι απλή αναδρομική και η FibonacciDP είναι αναδρομική με απομνημόνευση. Ο μετρητής cnt1 μετράει τις κλήσεις στην απλή αναδρομική και ο cnt2 μετράει τις κλήσεις στην αναδρομική με απομνημόνευση. Η έξοδος του κώδικα είναι:

```
fib(20)=10946
----------------------
fib(20)=10946
21891 39
```

Επομένως προκειμένου να υπολογισθεί ο 20ός όρος της ακολουθίας η απλή αναδρομική συνάρτηση πραγματοποιεί 21891 κλήσεις κι η αναδρομική με απομνημόνευση πραγματοποιεί 39 κλήσεις.

Η FibonacciDP αξιοποιεί ένα HashMap<Integer, Long>. Ο Integer σε αυτό το HashMap αντιπροσωπεύει την τάξη του όρου της ακολουθίας και ο Long την τιμή του αντίστοιχου όρου. Κατά την κλήση της συνάρτησης, γίνεται προσπάθεια να ανακτηθεί από το HashMap η τιμή του ζητούμενου όρου. Εφόσον αυτή δεν βρεθεί, παράγεται εξαίρεση. Κατά τη σύλληψη της εξαίρεσης υπολογίζεται ο όρος και εισάγεται στο HashMap, έτσι την επόμενη φορά που θα ζητηθεί ο ίδιος όρος θα βρεθεί στο HashMap και δεν θα απαιτηθεί ο επανυπολογισμός του.

18.3 Λυμένες ασκήσεις

18.3.1 Εφαρμογή εκφράσεων λάμδα

Δημιουργήστε τρεις εκφράσεις λάμδα. Η πρώτη θα υπολογίζει το άθροισμα των στοιχείων ενός πίνακα πραγματικών, η δεύτερη θα υπολογίζει τον μέσο όρο των στοιχείων ενός πίνακα πραγματικών και η τρίτη το ελάχιστο στοιχείο ενός πίνακα πραγματικών. Εκχωρήστε τις εκφράσεις λάμδα σε μεταβλητές ονομαζόμενες addition, average, minElement, αντίστοιχα. Προσθέστε main στην οποία υπολογίστε το άθροισμα, τον μέσο όρο και το ελάχιστο στοιχείο ενός πίνακα πραγματικών χρησιμοποιώντας τις αντίστοιχες μεταβλητές.

Λύση

```
interface ArrayProcessor {
    double apply( double[] array );
}

public class Processor {

static ArrayProcessor addition= array-> {
    double sum=0;
    for (double d: array)
        sum+=d;
    return sum;
};

static ArrayProcessor average=array-> {
    return addition.apply(array)/array.length;
};

static ArrayProcessor minElement=array-> {
    double min=array[0];
    for (int i=1; i<array.length; i++)
        if (array[i]<min) {
            min=array[i];
        }
    return min;
};

    public static void main(String[] args) {
        double[] array=new double[]{1,2,3,4};
        double sum=addition.apply(array);
        System.out.println(sum);
        System.out.println(average.apply(array));
        System.out.println(minElement.apply(array));
    }

}
```

Κώδικας 18.11 *Υπολογισμοί σε πίνακα πραγματικών με εκφράσεις λάμδα*

Καταρχάς, προκειμένου να ορίσουμε τις εκφράσεις λάμδα, θα πρέπει να έχουμε στη διάθεσή μας κατάλληλη λειτουργική διεπαφή, δηλαδή μία διεπαφή που δηλώνει μία αφηρημένη μέθοδο η οποία λαμβάνει ως παράμετρο έναν πίνακα πραγματικών και επιστρέφει έναν πραγματικό αριθμό. Πράγματι, η διεπαφή ArrayProcessor που δηλώνεται στην αρχή του κώδικα 18.11 πληροί τις απαιτούμενες προδιαγραφές.

Στη συνέχεια, ορίζουμε τις εκφράσεις λάμδα τις οποίες εκχωρούμε στις αντίστοιχες μεταβλητές των οποίων ο τύπος είναι ArrayProcessor. Στην main καλούμε διαμέσου των μεταβλητών την apply δίνοντάς της έναν πίνακα πραγματικών ως παράμετρο.

18.3.2 Αναδρομικός υπολογισμός δύναμης με απομνημόνευση

Υλοποιήστε μια συνάρτηση αναδρομικού υπολογισμού δύναμης με πραγματική βάση υψωμένη σε ακέραιο εκθέτη χωρίς απομνημόνευση και μία με απομνημόνευση. Μελετήστε πώς ακριβώς θα πρέπει να αναπτυχθεί η συνάρτηση με απομνημόνευση ώστε να έχουμε περιορισμό των αναδρομικών κλήσεων.

Λύση

```
class Tupple {

    double b;
    int e;

    public Tupple(double b, int e) {
        this.b = b;
        this.e = e;
    }

    @Override
    public int hashCode() {
        int hash = 3;
        hash = 43 * hash + (int) this.b;
        hash = 43 * hash + this.e;
        return hash;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (obj == null) {
            return false;
        }
        if (getClass() != obj.getClass()) {
            return false;
        }
        final Tupple other = (Tupple) obj;
        if (this.b != other.b) {
            return false;
        }
        return (int) this.e == (int) other.e;
    }

}

public class Power {

    static long cnt1 = 0;
    static long cnt2 = 0;
    static HashMap<Tupple, Double> mem = new HashMap<>();

    static double pow(int b, int e) {
        cnt1++;
        if (e > 0) {
            return b * pow(b, e - 1);
        } else if (e < 0) {
            return 1 / pow(b, -e);
        }
        return 1;
    }

    static double fastPow(int b, int e, HashMap<Tupple, Double> mem) {
        cnt2++;
        if (e != 0) {
            try {
                return mem.get(new Tupple(b, e));
```